##

PRACTICAL 1: ENVIRONMENT SETUP AND JAVASCRIPT BASICS

##

FILE: app.js (inside folder Pract1)

--- 1a. Install Node.js and VS Code, verify ---

STEPS:

Download Node.js from nodejs.org -> Install -> Download VS Code -> Install -> Open Terminal -> Run commands below

COMMANDS (terminal):

node -v

npm -v

code --version

(If PowerShell blocks scripts, run this once:)

Set-ExecutionPolicy RemoteSigned

--- 1b. Simple Node.js file ---

STEPS:

Open VS Code -> Open folder Pract1 -> New file app.js -> Paste code -> Save -> Terminal -> node app.js

CODE (app.js):

```
console.log("Hello, Node.js!");
```

RUN:

node app.js

OUTPUT:

Hello, Node.js!

--- 1c. Basic commands (console log, variables, arithmetic) ---

STEPS:

Open app.js -> Replace code with below -> Save -> node app.js

CODE (app.js):

```
// Console Output
console.log("JavaScript Basics");

// Variables
let name = "Pooja";
let age = 30;

// Display variables
console.log("Name:", name);
console.log("Age:", age);

// Arithmetic Operations
let a = 15;
let b = 5;

console.log("Addition:", a + b);
console.log("Subtraction:", a - b);
console.log("Multiplication:", a * b);
```

```
console.log("Division:", a / b);
console.log("Modulus:", a % b);
```

OUTPUT:

```
JavaScript Basics
Name: Pooja
Age: 30
Addition: 20
Subtraction: 10
Multiplication: 75
Division: 3
Modulus: 0
```

--- 1d. Variables and functions ---

STEPS:

Open app.js -> Replace code with below -> Save -> node app.js

CODE (app.js):

```
// Variables
let num1 = 10;
let num2 = 20;

// Function Definition
function add(x, y) {
  return x + y;
}

// Function Call
```

```
let result = add(num1, num2);
console.log("Sum =", result);

function greet(name) {
  return `Welcome ${name}!`;
}

console.log(greet("Pooja"));
```

OUTPUT:

Sum = 30

Welcome Pooja!

--- 1e. Operators and control statements ---

STEPS:

Open app.js -> Replace code with below -> Save -> node app.js

CODE (app.js):

```
let x = 12;
let y = 4;

console.log("Addition:", x + y);
console.log("Greater Than:", x > y);
console.log("Equal To:", x == y);
console.log("Logical AND:", (x > 5 && y < 10));

//If-Else Statement
let marks = 75;
```

```
if (marks >= 40) {
  console.log("Pass");
} else {
  console.log("Fail");
}

//Switch Statement
let day = 3;

switch(day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  default:
    console.log("Invalid Day");
}
```

OUTPUT:

Addition: 16

Greater Than: true

Equal To: false

Logical AND: true

Pass

Wednesday

--- 1f. Loops and template literals ---

STEPS:

Open app.js -> Replace code with below -> Save -> node app.js

CODE (app.js):

```
// for loop
for(let i = 1; i <= 5; i++) {
  console.log(i);
}

//while loop
let count = 1;

while(count <= 5) {
  console.log(count);
  count++;
}

//Template Literals
let student = "Pooja";
let score = 90;

console.log(`Student Name: ${student}`);
console.log(`Score: ${score}`);
```

OUTPUT:

1 2 3 4 5 (for loop)

1 2 3 4 5 (while loop)

Student Name: Pooja

Score: 90

##

PRACTICAL 2: CORE JAVASCRIPT CONCEPTS FOR MERN

##

FILE: app.js (inside folder Pract2)

STEPS:

Create folder Pract2 -> Open in VS Code -> New file app.js -> Paste code -> Save -> Terminal -> node app.js

CODE (app.js):

```
//a. Create functions and call them with parameters
// Function Declaration
function greet(name) {
  return "Hello, " + name + "!";
}

// Function Call
console.log(greet("Alice"));
console.log(greet("John"));
```

```
//b. Work with arrays and objects in JavaScript
let fruits = ["Apple", "Banana", "Mango"];
console.log(fruits[0]);
fruits.push("Orange");
console.log(fruits);

//c. Demonstrate use of arrow functions
const add = (a, b) => {
  return a + b;
};
console.log(add(10, 20));

//d. Write a program using array methods (map, filter, reduce)
let numbers = [10, 20, 30, 40, 50];

// map()
let doubled = numbers.map(num => num * 2);

// filter()
let greaterThan25 = numbers.filter(num => num > 25);

// reduce()
let total = numbers.reduce((sum, num) => sum + num, 0);

console.log("Original:", numbers);
console.log("Doubled:", doubled);
console.log("Greater than 25:", greaterThan25);
```

```
console.log("Sum:", total);

//e. Create a program that manipulates objects and displays output
let employee = {
  id: 101,
  name: "Amit",
  department: "IT",
  salary: 50000
};

// Update salary
employee.salary = 60000;

// Add new property
employee.city = "Mumbai";

// Delete department
delete employee.department;

console.log(employee);

//f. Implement a small JavaScript program combining functions, arrays, and objects
// Array of student objects
const students = [
  { name: "Alice", marks: 85 },
  { name: "Bob", marks: 72 },
  { name: "Charlie", marks: 90 },
  { name: "David", marks: 65 }
```

```
];

// Function to calculate average marks
const averageMarks = (students) => {
  const total = students.reduce((sum, student) => sum + student.marks, 0);
  return total / students.length;
};

// Function to find passed students
const passedStudents = (students) => {
  return students.filter(student => student.marks >= 75);
};

console.log("Average Marks:", averageMarks(students));

console.log("Passed Students:");
passedStudents(students).forEach(student => {
  console.log(`${student.name} - ${student.marks}`);
});
```

RUN:

node app.js

OUTPUT:

Hello, Alice!

Hello, John!

Apple

['Apple', 'Banana', 'Mango', 'Orange']

30

Original: [10, 20, 30, 40, 50]

Doubled: [20, 40, 60, 80, 100]

Greater than 25: [30, 40, 50]

Sum: 150

{ id: 101, name: 'Amit', salary: 60000, city: 'Mumbai' }

Average Marks: 78

Passed Students:

Alice - 85

Charlie - 90

##

PRACTICAL 3: REACT INTRODUCTION AND JSX

##

FILES: src/App.jsx, src/App.css, src/Greeting.jsx, src/DateTime.jsx

(App.jsx is replaced for each part a to f. Greeting.jsx, DateTime.jsx and App.css are new files)

--- 3a. Create React app and display message ---

STEPS:

Open terminal -> Run commands below -> Browser opens automatically -> Open src/App.jsx -> Delete default code -> Paste code -> Save

COMMANDS:

npx create-react-app pract3app

cd pract3app

npm start

(Open in browser: localhost:3000 for CRA, localhost:5173 for Vite)

CODE (src/App.jsx):

```
function App() {
  return (
    <div>
      <h1>Welcome to MERN Stack Development</h1>
    </div>
  );
}

export default App;
```

--- 3b. Display dynamic data using JSX ---

STEPS:

Open src/App.jsx -> Replace code with below -> Save -> Check browser

CODE (src/App.jsx):

```
function App() {
  const name = "Pooja";
  const course = "MERN Stack Development";

  return (
    <div>
      <h2>Hello, {name}</h2>
      <p>Welcome to {course}</p>
    </div>
```

```
  );
}

export default App;
```

--- 3c. Sum of two numbers ---

STEPS:

Open src/App.jsx -> Replace code with below -> Save -> Check browser

CODE (src/App.jsx):

```
function App() {
  const num1 = 25;
  const num2 = 15;

  function calculateSum(a, b) {
    return a + b;
  }

  return (
    <div>
      <h2>Sum Calculation</h2>
      <p>Number 1: {num1}</p>
      <p>Number 2: {num2}</p>
      <p>Sum: {calculateSum(num1, num2)}</p>
    </div>
  );
}
```

export default App;

--- 3d. Basic CSS styling ---

STEPS:

Open src/App.css -> Replace code with CSS below -> Open src/App.jsx -> Replace code with JSX below -> Save -> Check browser

CODE (src/App.css):

```
.container {
  text-align: center;
  background-color: lightblue;
  padding: 20px;
  margin: 20px;
  border-radius: 10px;
}

h1 {
  color: navy;
}

p {
  color: darkgreen;
  font-size: 18px;
}
```

CODE (src/App.jsx):

```
import "./App.css";
```

```
function App() {
  return (
    <div className="container">
      <h1>React CSS Example</h1>
      <p>This component is styled using CSS.</p>
    </div>
  );
}

export default App;
```

--- 3e. Simple functional component ---

STEPS:

Create new file src/Greeting.jsx -> Paste first code -> Open src/App.jsx -> Replace with second code -> Save -> Check browser

CODE (src/Greeting.jsx):

```
function Greeting() {
  return (
    <div>
      <h2>Hello from Greeting Component!</h2>
    </div>
  );
}

export default Greeting;
```

CODE (src/App.jsx):

```
import Greeting from "./Greeting";

function App() {
  return (
    <div>
      <h1>Main App Component</h1>
      <Greeting />
    </div>
  );
}

export default App;
```

--- 3f. Current date and time ---

STEPS:

Create new file src/DateTime.jsx -> Paste first code -> Open src/App.jsx -> Replace with second code -> Save -> Check browser

CODE (src/DateTime.jsx):

```
function DateTime() {
  const currentDate = new Date();

  return (
    <div>
      <h2>Current Date and Time</h2>
      <p>Date: {currentDate.toLocaleDateString()}</p>
      <p>Time: {currentDate.toLocaleTimeString()}</p>
    </div>
```

```
  );
}

export default DateTime;
```

CODE (src/App.jsx):

```
import DateTime from "./DateTime";

function App() {
  return (
    <div>
      <h1>Date and Time Example</h1>
      <DateTime />
    </div>
  );
}

export default App;
```

##

PRACTICAL 4: REACT COMPONENTS, PROPS, AND STATE

##

FILES: src/App.jsx (replaced for each part), src/Student.jsx (Part B only)

SETUP STEPS:

Open terminal -> Run commands below -> Open src/App.jsx for each part

COMMANDS:

npx create-react-app pract4app

cd pract4app

npm start

--- 4a. Multiple components on a single page ---

STEPS:

Open src/App.jsx -> Replace code with below -> Save -> Check browser

CODE (src/App.jsx):

```
import React from "react";

// Component 1
function Header() {
  return <h1>Welcome to React</h1>;
}

// Component 2
function Student() {
  return <h2>Student Information</h2>;
}

// Component 3
function Course() {
  return <p>Course: MERN Stack Development</p>;
}
```

```
// Main Component
function App() {
  return (
    <div>
      <Header />
      <Student />
      <Course />
    </div>
  );
}

export default App;
```

--- 4b. Pass data using props ---

STEPS:

Create new file src/Student.jsx -> Paste first code -> Open src/App.jsx -> Replace with second code -> Save

CODE (src/Student.jsx):

```
function Student(props) {
  return (
    <div>
      <h2>Student Details</h2>
      <p>Name: {props.name}</p>
      <p>Course: {props.course}</p>
    </div>
  );
}
```

```
export default Student;
```

CODE (src/App.jsx):

```
import Student from "./Student";

function App() {
  return (
    <div>
      <h1>React Props Example</h1>
      <Student
        name="Pooja"
        course="MERN Stack Development"
      />
    </div>
  );
}

export default App;
```

--- 4c. useState hook (counter) ---

STEPS:

Open src/App.jsx -> Replace code with below -> Save

CODE (src/App.jsx):

```
import React, { useState } from "react";

function App() {
```

```
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Counter</h1>
      <h2>Count: {count}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>

      <button onClick={() => setCount(count - 1)}>
        Decrease
      </button>

      <button onClick={() => setCount(0)}>
        Reset
      </button>
    </div>
  );
}

export default App;
```

--- 4d. Form to accept input and display dynamically ---

STEPS:

Open src/App.jsx -> Replace code with below -> Save

CODE (src/App.jsx):

```
import React, { useState } from "react";

function App() {
  const [name, setName] = useState("");

  return (
    <div>
      <h1>Student Form</h1>

      <input
        type="text"
        placeholder="Enter your name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <h2>Hello, {name}</h2>
    </div>
  );
}

export default App;
```

--- 4e. Display list using map() ---

STEPS:

Open src/App.jsx -> Replace code with below -> Save

CODE (src/App.jsx):

```
import React, { useState } from "react";

function App() {
  const [students] = useState([
    "Pooja",
    "Rahul",
    "Sneha",
    "Amit"
  ]);

  return (
    <div>
      <h1>Student List</h1>

      <ul>
        {students.map((student, index) => (
          <li key={index}>
            {student}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

--- 4f. To-Do list (interactive component) ---

STEPS:

Open src/App.jsx -> Replace code with below -> Save

CODE (src/App.jsx):

```
import React, { useState } from "react";

function App() {
  const [task, setTask] = useState("");
  const [todos, setTodos] = useState([]);

  // Add task
  const addTask = () => {
    if (task !== "") {
      setTodos([...todos, task]);
      setTask("");
    }
  };

  // Delete task
  const deleteTask = (index) => {
    setTodos(todos.filter((w, i) => i !== index));
  };

  return (
    <div>
      <h1>My To-Do List</h1>
```

```
      <input
        type="text"
        placeholder="Enter a task"
        value={task}
        onChange={(e) => setTask(e.target.value)}
      />

      <button onClick={addTask}>
        Add Task
      </button>

      <h2>Tasks</h2>

      <ul>
        {todos.map((todo, index) => (
          <li key={index}>
            {todo}
            <button onClick={() => deleteTask(index)}>
              Delete
            </button>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

##

PRACTICAL 5: REACT ROUTING AND DATA HANDLING

##

FILES: src/App.js, src/pages/Home.js, src/pages/About.js, src/pages/Users.js

(create folder "pages" inside src)

STEPS:

Open terminal -> Run commands below -> Create folder src/pages -> Create Home.js, About.js, Users.js -> Paste codes -> Replace App.js -> Save -> Open localhost:3000

COMMANDS:

npx create-react-app react-routing-app

cd react-routing-app

npm install react-router-dom axios

npm start

CODE (src/App.js):

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

import Home from "./pages/Home";
import About from "./pages/About";
import Users from "./pages/Users";

function App() {
  return (
```

```
    <BrowserRouter>
      <div>
        <nav>
          <ul style={{ display: "flex", gap: "20px" }}>
            <li>
              <Link to="/">Home</Link>
            </li>
            <li>
              <Link to="/about">About</Link>
            </li>
            <li>
              <Link to="/users">Users</Link>
            </li>
          </ul>
        </nav>

        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          <Route path="/users" element={<Users />} />
        </Routes>
      </div>
    </BrowserRouter>
  );
}

export default App;
```

CODE (src/pages/Home.js):

```
function Home() {
  return (
    <div>
      <h1>Home Page</h1>
      <p>Welcome to React Router Demo</p>
    </div>
  );
}

export default Home;
```

CODE (src/pages/About.js):

```
function About() {
  return (
    <div>
      <h1>About Page</h1>
      <p>This application demonstrates routing and API fetching.</p>
    </div>
  );
}

export default About;
```

CODE (src/pages/Users.js):

```
import axios from "axios";
import { useEffect, useState } from "react";
```

```
function Users() {
  const [users, setUsers] = useState([]);
  const [search, setSearch] = useState("");

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      })
      .catch((error) => {
        console.log(error);
      });
  }, []);

  const filteredUsers = users.filter((user) =>
    user.name.toLowerCase().includes(search.toLowerCase())
  );

  return (
    <div>
      <h1>Users List</h1>

      <input
        type="text"
        placeholder="Search User..."
        value={search}
        onChange={(e) => setSearch(e.target.value)}
```

```
      />

      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>City</th>
          </tr>
        </thead>

        <tbody>
          {filteredUsers.map((user) => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.email}</td>
              <td>{user.address.city}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default Users;
```

TEST:

localhost:3000 -> Home | localhost:3000/about -> About | localhost:3000/users -> table, type a name in the search box to filter

##

PRACTICAL 6: NODE.JS AND EXPRESS FUNDAMENTALS

##

FILE: server.js (inside folder Pract6)

STEPS:

Create folder Pract6 -> Open terminal in it -> npm install express -> New file server.js -> Paste code -> Save -> node server.js -> Open browser

COMMANDS:

npm install express

node server.js

CODE (server.js):

```
const express = require('express');

const app = express();

const PORT = 3000;

// Home Route

app.get('/', (req, res) => {

  res.send('Welcome to Home Page');

});
```

```
// About Route
app.get('/about', (req, res) => {
  res.send('About Express Application');
});

// Contact Route
app.get('/contact', (req, res) => {
  res.send('Contact Us');
});

// JSON Route
app.get('/student', (req, res) => {
  res.json({
    id: 1,
    name: 'Rahul',
    age: 21,
    course: 'Node.js'
  });
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

TEST (browser or Postman, GET method):

http://localhost:3000/ -> Welcome to Home Page

http://localhost:3000/about -> About Express Application

http://localhost:3000/contact -> Contact Us

http://localhost:3000/student -> {"id":1,"name":"Rahul","age":21,"course":"Node.js"}

CONSOLE OUTPUT:

Server running at http://localhost:3000

##

PRACTICAL 7: MONGODB BASICS

##

FILE: app.js (inside folder Pract7, for 7f only)

Other parts are done in MongoDB Compass and MONGOSH.

--- 7a. Install and start MongoDB ---

STEPS:

Download MongoDB Community Server -> Install (also install MongoDB Compass) -> Open MongoDB Compass -> Click Connect (localhost:27017)

--- 7b. Create database and collection ---

STEPS:

Compass -> Click "Create database" -> Database Name: emp -> Collection Name: e1 -> Click "Create Database"

--- 7c. Insert records ---

STEPS:

Open emp -> e1 -> ADD DATA -> Insert document -> Paste below -> Click Insert

Repeat for B (clerk, 35000), C (manager, 74000), D (manager, 79000)

CODE (Insert Document):

```
{
  "emp_nm":"A",
  "emp_desg":"clerk",
  "emp_sal":34000
}
```

--- 7d. View records ---

STEPS:

Open e1 collection -> All documents are listed under Documents tab

MONGOSH COMMANDS:

```
use emp
db.e1.find({},{})
db.e1.find({"emp_nm":"C"},{})
db.e1.find({},{"emp_nm":1,"emp_desg":1})
```

--- 7e. Update and delete records ---

STEPS (Compass):

Hover on document A -> Click pencil (Edit document) -> Change emp_sal 34000 to 34500 -> Click UPDATE

Hover on document D -> Click bin (Remove document) -> Confirm Delete

MONGOSH COMMANDS (insert, update, delete):

```
use emp
db.e1.insertOne({"emp_nm":"D","emp_desg":"manager","emp_sal":70000})
db.e1.updateMany({"emp_desg":"manager"},{"$set":{"emp_sal":80000}})
db.e1.find({},{})
```

db.e1.deleteMany({"emp_sal":80000},{})

db.e1.find({},{})

--- 7f. Connect MongoDB with Node.js ---

STEPS:

Create folder Pract7 -> Open terminal -> npm install mongoose mongodb -> New file app.js -> Paste code -> Save -> node app.js

COMMANDS:

npm install mongoose mongodb

node app.js

CODE (app.js):

```
// npm install mongoose mongodb
const mongoose = require('mongoose');

//Connection URI to Mongoose
const uri = 'mongodb://127.0.0.1:27017/emp';

//Make db connection (Asynchronously)
mongoose.connect(uri)
.then (result =>
{ console.log('Successful connection!!.....')
})
.catch( error => console.log(error));
```

OUTPUT:

Successful connection!!.....

##

PRACTICALS 8, 9, 10 - HOW TO RUN (COMMON)

##

STEPS:

Make sure MongoDB is running

-> Terminal 1: go to folder containing server.js (src folder) -> node server.js (keep it running)

-> Terminal 2: go to React app folder -> npm start

-> Browser opens React app (localhost:3000)

COMMANDS (install once in the app folder):

npm install express mongoose cors

Create React apps:

Pract8: npx create-react-app pract8app

Pract9: npx create-react-app pract9app

Pract10: npx create-react-app pract10app

##

PRACTICAL 8: API CREATION AND INTEGRATION

##

FILES: pract8app/src/server.js, pract8app/src/App.jsx

(Database: studentDB)

STEPS:

npx create-react-app pract8app -> cd pract8app -> npm install express mongoose cors -> Create src/server.js -> Replace src/App.jsx -> Terminal 1: cd src, node server.js -> Terminal 2: npm start

CODE (src/server.js):

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

// Connect to MongoDB
mongoose
  .connect("mongodb://127.0.0.1:27017/studentDB")
  .then(() => console.log("MongoDB Connected"))
  .catch((err) => console.log(err));

// Schema
const studentSchema = new mongoose.Schema({
  name: String,
  age: Number
});

// Model
const Student = mongoose.model("Student", studentSchema);
```

```
// GET API - retrieve data
app.get("/students", async (req, res) => {
  const students = await Student.find();
  res.json(students);
});

// POST API - receive and store data
app.post("/students", async (req, res) => {
  const student = new Student({
    name: req.body.name,
    age: req.body.age
  });

  await student.save();

  res.json({
    message: "Student saved successfully",
    student: student
  });
});

// Start server
app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

CODE (src/App.jsx):

```
import { useEffect, useState } from "react";

function App() {
  const [students, setStudents] = useState([]);
  const [name, setName] = useState("");
  const [age, setAge] = useState("");

  // GET data from API
  const getStudents = async () => {
    const response = await fetch("http://localhost:5000/students");
    const data = await response.json();
    setStudents(data);
  };

  // POST data to API
  const addStudent = async (e) => {
    e.preventDefault();

    await fetch("http://localhost:5000/students", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        name: name,
        age: Number(age)
      })
    });
```

```
  setName("");
  setAge("");

  // Refresh displayed data
  getStudents();
};

// Get students when page loads
useEffect(() => {
  getStudents();
}, []);

return (
  <div>
    <h1>Student Management</h1>

    <form onSubmit={addStudent}>
      <input
        type="text"
        placeholder="Enter name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <input
        type="number"
        placeholder="Enter age"
```

```
          value={age}
          onChange={(e) => setAge(e.target.value)}
        />

        <button type="submit">Add Student</button>
      </form>

      <h2>Students</h2>

      {students.map((student) => (
        <p key={student._id}>
          {student.name} - {student.age} years
        </p>
      ))}
    </div>
  );
}

export default App;
```

OUTPUT:

Server console: MongoDB Connected / Server running on port 5000

Browser: Student Management form + list (e.g. AA - 12 years)

##

PRACTICAL 9: USER DATA MANAGEMENT

##

FILES: pract9app/src/server.js, pract9app/src/App.jsx

(Database: userDB)

STEPS:

npx create-react-app pract9app -> cd pract9app -> npm install express mongoose cors -> Create src/server.js -> Replace src/App.jsx -> Terminal 1: cd src, node server.js -> Terminal 2: npm start

CODE (src/server.js):

```
const express = require("express");

const mongoose = require("mongoose");

const cors = require("cors");


const app = express();


app.use(cors());

app.use(express.json());


// Connect MongoDB

mongoose

  .connect("mongodb://127.0.0.1:27017/userDB")

  .then(() => console.log("MongoDB Connected"))

  .catch((err) => console.log(err));


// User Schema

const userSchema = new mongoose.Schema({

  name: String,

  email: String,
```

```
  age: Number
});

const User = mongoose.model("User", userSchema);

// ADD USER
app.post("/users", async (req, res) => {
  const user = new User(req.body);
  await user.save();
  res.json(user);
});

// GET USERS
app.get("/users", async (req, res) => {
  const users = await User.find();
  res.json(users);
});

// UPDATE USER
app.put("/users/:id", async (req, res) => {
  const user = await User.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true }
  );
  res.json(user);
});
```

```
// DELETE USER
app.delete("/users/:id", async (req, res) => {
  await User.findByIdAndDelete(req.params.id);
  res.json({ message: "User deleted" });
});

// Start server
app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

CODE (src/App.jsx):

```
import { useEffect, useState } from "react";

function App() {
  const [users, setUsers] = useState([]);
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [age, setAge] = useState("");
  const [editId, setEditId] = useState(null);

  // GET USERS
  const getUsers = async () => {
    const response = await fetch("http://localhost:5000/users");
    const data = await response.json();
    setUsers(data);
  };
```

```
// ADD / UPDATE USER
const saveUser = async (e) => {
  e.preventDefault();

  const userData = {
    name: name,
    email: email,
    age: Number(age)
  };

  if (editId) {
    // UPDATE
    await fetch(`http://localhost:5000/users/${editId}`, {
      method: "PUT",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(userData)
    });

    setEditId(null);
  } else {
    // ADD
    await fetch("http://localhost:5000/users", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
```

```
      body: JSON.stringify(userData)
    });
  }

  clearForm();
  getUsers();
};

// DELETE USER
const deleteUser = async (id) => {
  await fetch(`http://localhost:5000/users/${id}`, {
    method: "DELETE"
  });

  getUsers();
};

// EDIT USER
const editUser = (user) => {
  setName(user.name);
  setEmail(user.email);
  setAge(user.age);
  setEditId(user._id);
};

// CLEAR FORM
const clearForm = () => {
  setName("");
```

```
  setEmail("");
  setAge("");
};

// Load users when page opens
useEffect(() => {
  getUsers();
}, []);

return (
  <div>
    <h1>User Data Management</h1>

    <form onSubmit={saveUser}>
      <input
        type="text"
        placeholder="Name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <input
        type="email"
        placeholder="Email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
```

```
  <input
    type="number"
    placeholder="Age"
    value={age}
    onChange={(e) => setAge(e.target.value)}
  />

  <button type="submit">
    {editId ? "Update User" : "Add User"}
  </button>
</form>

<h2>User List</h2>

{users.map((user) => (
  <div key={user._id}>
    <p>
      <b>{user.name}</b> - {user.email} - {user.age} years
    </p>

    <button onClick={() => editUser(user)}>
      Edit
    </button>

    <button onClick={() => deleteUser(user._id)}>
      Delete
    </button>
  </div>
```

```
    ))}
  </div>
 );
}

export default App;
```

##

PRACTICAL 10: FULL STACK EXECUTION AND VALIDATION

##

FILES: pract10app/src/server.js, pract10app/src/App.jsx

(Database: userDB)

STEPS:

npx create-react-app pract10app -> cd pract10app -> npm install express mongoose cors -> Create src/server.js -> Replace src/App.jsx -> Terminal 1: cd src, node server.js -> Terminal 2: npm start

CODE (src/server.js):

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());
```

```
// MongoDB connection
mongoose
  .connect("mongodb://127.0.0.1:27017/userDB")
  .then(() => console.log("MongoDB Connected"))
  .catch((err) => console.log(err));

// User Schema
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});

const User = mongoose.model("User", userSchema);

// GET - Display users
app.get("/users", async (req, res) => {
  const users = await User.find();
  res.json(users);
});

// POST - Add user
app.post("/users", async (req, res) => {
  const user = new User(req.body);
  await user.save();
  res.json(user);
});
```

```
// PUT - Update user
app.put("/users/:id", async (req, res) => {
  const user = await User.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true }
  );
  res.json(user);
});

// DELETE - Delete user
app.delete("/users/:id", async (req, res) => {
  await User.findByIdAndDelete(req.params.id);
  res.json({ message: "User deleted successfully" });
});

// Start server
app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

CODE (src/App.jsx):

```
import { useEffect, useState } from "react";

function App() {
  const [users, setUsers] = useState([]);
  const [name, setName] = useState("");
```

```
const [email, setEmail] = useState("");
const [age, setAge] = useState("");
const [editId, setEditId] = useState(null);
const [error, setError] = useState("");

// Get users
const getUsers = async () => {
  const response = await fetch("http://localhost:5000/users");
  const data = await response.json();
  setUsers(data);
};

// Form validation
const validateForm = () => {
  if (name.trim() === "") {
    setError("Name is required");
    return false;
  }

  if (!email.includes("@")) {
    setError("Enter a valid email");
    return false;
  }

  if (age <= 0) {
    setError("Age must be greater than 0");
    return false;
  }
```

```
  setError("");
  return true;
};

// Add or Update
const saveUser = async (e) => {
  e.preventDefault();

  if (!validateForm()) {
    return;
  }

  const userData = {
    name: name,
    email: email,
    age: Number(age)
  };

  if (editId) {
    // UPDATE
    await fetch(`http://localhost:5000/users/${editId}`, {
      method: "PUT",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(userData)
    });
```

```
    setEditId(null);
  } else {
    // INSERT
    await fetch("http://localhost:5000/users", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(userData)
    });
  }

  clearForm();

  // Refresh data dynamically
  getUsers();
};

// Delete
const deleteUser = async (id) => {
  await fetch(`http://localhost:5000/users/${id}`, {
    method: "DELETE"
  });

  getUsers();
};
```

```
// Edit
const editUser = (user) => {
  setName(user.name);
  setEmail(user.email);
  setAge(user.age);
  setEditId(user._id);
};

// Clear form
const clearForm = () => {
  setName("");
  setEmail("");
  setAge("");
  setError("");
};

// Load users when page starts
useEffect(() => {
  getUsers();
}, []);

return (
  <div>
    <h1>User Management</h1>

    <form onSubmit={saveUser}>
      <input
        type="text"
```

```
  placeholder="Name"
  value={name}
  onChange={(e) => setName(e.target.value)}
/>

<br /><br />

<input
  type="email"
  placeholder="Email"
  value={email}
  onChange={(e) => setEmail(e.target.value)}
/>

<br /><br />

<input
  type="number"
  placeholder="Age"
  value={age}
  onChange={(e) => setAge(e.target.value)}
/>

<br /><br />

<button type="submit">
  {editId ? "Update User" : "Add User"}
</button>
```

```
      {error && <p style={{ color: "red" }}>{error}</p>}
    </form>

    <h2>Users</h2>

    {users.map((user) => (
      <div key={user._id}>
        <p>
          {user.name} | {user.email} | {user.age}
        </p>

        <button onClick={() => editUser(user)}>
          Edit
        </button>

        <button onClick={() => deleteUser(user._id)}>
          Delete
        </button>

        <hr />
      </div>
    ))}
  </div>
 );
}

export default App;
```

--- 10e. Test APIs using Postman ---

STEPS:

Keep server.js running -> Open Postman -> Test each request below

GET http://localhost:5000/users -> Send -> list of users

POST http://localhost:5000/users -> Body -> raw -> JSON:

{"name":"Pooja","email":"pooja@gmail.com","age":21}

PUT http://localhost:5000/users/<id> -> Body -> raw -> JSON:

{"name":"Pooja S","email":"pooja@gmail.com","age":22}

DELETE http://localhost:5000/users/<id> -> Send -> {"message":"User deleted successfully"}

(Replace <id> with the _id value from the GET response.)

--- 10f. Verify complete working ---

STEPS:

Open React page -> Add user -> Appears in list -> Click Edit -> Change -> Update User -> Click Delete -> Record removed -> Leave Name empty / email without @ / age 0 -> Red error shows -> Check records in MongoDB Compass (userDB -> users collection)